

IST602: Web Technologies and Standards - Practice Exam Sets

Course Code: IST602

Institution: University of Buea, Faculty of Science

Level: Master's Program

Time Allowed: 3 Hours per Exam

PRACTICE EXAM SET 1

Total Marks: 100

SECTION A: MULTIPLE CHOICE QUESTIONS (20 Marks)

(1 mark each)

1. Which of the following is a correct statement about TCP?

- a) Connectionless, unreliable
- b) Connection-oriented, reliable and ordered
- c) Lightweight, no error checking
- d) Used for real-time streaming only

Answer: b

Explanation: TCP (Transmission Control Protocol) is connection-oriented, provides reliable and ordered delivery, with error checking and retransmission. UDP is connectionless and used for real-time streaming.

2. What is the primary purpose of XML namespaces?

- a) To make XML files smaller
- b) To prevent element name collisions between different vocabularies
- c) To encrypt XML content
- d) To convert XML to HTML

Answer: b

Explanation: XML namespaces allow mixing different XML vocabularies (like XHTML and SVG) in one document without element name conflicts.

3. In CSS, which selector targets all `<p>` elements inside `<div>` elements?

- a) `p.div`
- b) `div p`
- c) `div + p`
- d) `p > div`

Answer: b

Explanation: `div p` is a descendant selector. `div > p` would be a direct child selector.

4. What will be the output of `typeof null` in JavaScript?

- a) `"null"`
- b) `"object"`
- c) `"undefined"`
- d) `"boolean"`

Answer: b

Explanation: A classic JavaScript quirk! This is a historical bug that can't be fixed without breaking existing code.

5. In TypeScript, which keyword is used to define a reusable object shape contract?

- a) `class`
- b) `type`
- c) `interface`
- d) `struct`

Answer: c

Explanation: Interfaces define the structure (shape) that objects must conform to.

6. Which XML document property ensures exactly one top-level element?

- a) Validity
- b) Well-formedness
- c) Namespace
- d) Schema

Answer: b

Explanation: Well-formed XML requires exactly one root element. Valid XML also conforms to a schema/DTD.

7. What does the `*` operator mean in DTD content models?

- a) Zero or one occurrence
- b) Exactly one occurrence
- c) Zero or more occurrences
- d) One or more occurrences

Answer: c

Explanation: Remember: `?` = zero or one, `*` = zero or more, `+` = one or more.

8. In JavaScript, what does `parseInt("123abc")` return?

- a) `NaN`
- b) `123`
- c) `"123abc"`
- d) `undefined`

Answer: b

Explanation: `parseInt()` parses until it hits a non-digit character and returns what it found.

9. Which Bootstrap class creates a responsive fixed-width container?

- a) `container-fluid`
- b) `container`
- c) `container-box`
- d) `container-fixed`

Answer: b

Explanation: `container` = responsive fixed-width; `container-fluid` = full viewport width.

10. In TypeScript, what is the type of a function that doesn't return a value?

- a) `undefined`
- b) `null`
- c) `void`
- d) `any`

Answer: c

11. Which HTTP method is typically used to retrieve data from a server?

- a) POST
- b) GET
- c) PUT
- d) DELETE

Answer: b

12. What is a key difference between HTML and XML?

- a) HTML uses tags; XML doesn't
- b) XML is for display; HTML is for data
- c) XML allows custom tags; HTML has predefined tags
- d) HTML is strict; XML is forgiving

Answer: c

13. In JavaScript, which method adds an event listener to an element?

- a) `attachEvent()`
- b) `addEventListener()`
- c) `on()`
- d) `bindEvent()`

Answer: b

14. In TypeScript, which type allows values of multiple types?

- a) `any`
- b) `union`
- c) `mixed`
- d) `multi`

Answer: b

Explanation: Union type syntax: `string | number`.

15. What is the default sorting behavior of JavaScript's `array.sort()`?

- a) Numeric ascending
- b) String Unicode code point order
- c) Numeric descending
- d) Random order

Answer: b

Explanation: This is a common pitfall! You must provide a comparator function for numeric sorting.

16. Which CSS property controls the space inside an element's border?

- a) `margin`
- b) `padding`
- c) `border-spacing`
- d) `space`

Answer: b

17. In TypeScript, what is a tuple?

- a) A dynamic array
- b) A fixed-length array with known types at each position
- c) A key-value pair structure
- d) A set of unique values

Answer: b

18. What is the purpose of the XML declaration?

- a) To specify the root element
- b) To define the DTD
- c) To specify XML version and character encoding
- d) To link to CSS stylesheets

Answer: c

19. In Angular, what does the `@Component` decorator's `selector` property define?

- a) The component's CSS styling
- b) The HTML tag name used to embed the component
- c) The component's data model
- d) The component's dependencies

Answer: b

20. Which protocol is used to transfer web pages between client and server?

- a) FTP
- b) SMTP
- c) HTTP
- d) TCP

Answer: c

SECTION B: STRUCTURAL QUESTIONS (40 Marks)

Question 1: Web Fundamentals (10 marks)

a) Compare and contrast TCP and UDP. Provide two use cases for each. (6 marks)

Answer:

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Reliable, acknowledges, retransmits	Unreliable
Ordering	Guaranteed in-order delivery	No ordering
Overhead	Higher	Lower

TCP Use Cases: Web browsing (HTTP), file transfer (FTP)

UDP Use Cases: Video streaming, online gaming

b) Explain the client-server model in the context of the World Wide Web. (4 marks)

Answer:

- **Client:** Web browser that initiates HTTP requests
- **Server:** Stores web pages and responds to requests
- **Flow:** Browser → HTTP request → Server → HTTP response → Browser displays page

Question 2: XML and DTD (15 marks)

Given the following DTD:

```
<!ELEMENT library (book+)>
<!ELEMENT book (title, author+, year, price?)>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT price (#PCDATA)>
<!ATTLIST book isbn CDATA #REQUIRED>
```

a) Create a well-formed and valid XML document that conforms to this DTD, containing at least two books (one with price, one without). (8 marks)

Answer:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE library [
  <!ELEMENT library (book+)>
  <!ELEMENT book (title, author+, year, price?)>
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT author (#PCDATA)>
  <!ELEMENT year (#PCDATA)>
  <!ELEMENT price (#PCDATA)>
  <!-- ATTLIST book isbn CDATA #REQUIRED -->
]>
<library>
  <book isbn="978-0134685991">
    <title>Clean Code</title>
    <author>Robert C. Martin</author>
    <year>2008</year>
    <price>49.99</price>
  </book>
  <book isbn="978-0262033848">
    <title>AI: A Modern Approach</title>
    <author>Stuart Russell</author>
    <author>Peter Norvig</author>
    <year>2020</year>
  </book>
</library>
```

b) Explain the meaning of each of the following in DTD:

- i) **(book+)** (2 marks)
- ii) **price?** (2 marks)
- iii) **#REQUIRED** (3 marks)

Answer:

- i) **(book+)**: One or more **book** elements must appear
- ii) **price?**: The **price** element is optional (zero or one occurrence)
- iii) **#REQUIRED**: The **isbn** attribute is mandatory on every **book**

Question 3: CSS and Bootstrap (15 marks)

a) Explain the three ways to apply CSS to HTML documents, and discuss the advantages and disadvantages of each. (9 marks)

Answer:

1. **Inline Styles** (**style** attribute):

- Quick for testing
- Poor separation of concerns; not recommended for production

2. **Embedded Styles** (<style> in <head>):

- Good for single-page styles
- Not reusable across multiple pages

3. **External Stylesheets** (<link rel="stylesheet">):

- Best practice! Reusable, clean separation, cacheable
- Requires an extra HTTP request (negligible)

b) Write a Bootstrap HTML structure that creates:

- A responsive grid with 3 equal columns on large screens
- The columns stack vertically on small screens
- A **container** wrapper
- Each column has a background color (use Bootstrap utility classes) (6 marks)

Answer:

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <div class="container mt-4">
    <div class="row">
      <div class="col-lg-4 bg-primary text-white p-3">Column
1</div>
      <div class="col-lg-4 bg-secondary text-white p-3">Column
2</div>
      <div class="col-lg-4 bg-success text-white p-3">Column
3</div>
    </div>
  </div>
</body>
</html>
```

SECTION C: ESSAY AND CODE QUESTIONS (40 Marks)

Question 1: JavaScript Programming (20 marks)

a) Consider the following JavaScript code:

```
function calculate() {
  let numbers = [10, 5, 8, 3, 1];
  let sum = 0;
  let product = 1;

  for (let i = 0; i < numbers.length; i++) {
    sum += numbers[i];
    product *= numbers[i];
  }

  let average = sum / numbers.length;
  numbers.sort();

  return {
    sum: sum,
    product: product,
    average: average,
    sorted: numbers
  };
}

let result = calculate();
console.log(result.sorted);
```

i) What will be printed to the console? Explain why this happens and how to fix it. (8 marks)

Answer:

- **Output:** [1, 10, 3, 5, 8]
- **Why?** JavaScript's `sort()` converts elements to strings by default and compares their Unicode values. "10" comes before "3" because "1" has a lower Unicode value than "3".
- **Fix:** Use a comparator function.

ii) Rewrite the `sort()` call to properly sort the numbers in ascending order. (4 marks)

Answer:

```
numbers.sort((a, b) => a - b);
```

b) Write a complete HTML and JavaScript program that:

- Displays two input fields for numbers
- Has a "Calculate" button
- When clicked, shows the sum, difference, product, and quotient of the two numbers
- Handles division by zero gracefully (8 marks)

Answer:


```
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: Arial, sans-serif; max-width: 400px; margin: 2rem
auto; }
  </style>
</head>
<body>
  <h1>Simple Calculator</h1>
  <input type="number" id="num1" placeholder="First number">
  <input type="number" id="num2" placeholder="Second number">
  <button id="calculate">Calculate</button>
  <div id="result"></div>

  <script>
    document.getElementById('calculate').addEventListener('click',
function() {
      const num1 = parseFloat(document.getElementById('num1').value);
      const num2 = parseFloat(document.getElementById('num2').value);
      const resultDiv = document.getElementById('result');

      if (isNaN(num1) || isNaN(num2)) {
        resultDiv.innerHTML = 'Please enter valid numbers';
        return;
      }

      let quotient = num2 !== 0 ? num1 / num2 : 'Cannot divide by zero';

      resultDiv.innerHTML = `
        Sum: ${num1 + num2}<br>
        Difference: ${num1 - num2}<br>
        Product: ${num1 * num2}<br>
        Quotient: ${quotient}
      `;
    });
  </script>
</body>
</html>
```

Question 2: TypeScript and Angular (20 marks)

a) Define a TypeScript interface `Product` with the following properties:

- `id`: number (required)
- `name`: string (required)
- `price`: number (required)
- `description`: string (optional)
- `inStock`: boolean (required)

Then write a function `filterInStockProducts` that takes an array of `Product` objects and returns only those that are in stock. (10 marks)

Answer:

```
interface Product {
  id: number;
  name: string;
  price: number;
  description?: string;
  inStock: boolean;
}

function filterInStockProducts(products: Product[]): Product[] {
  return products.filter(p => p.inStock);
}
```

b) Briefly explain the role of the following in Angular:

- i) Component metadata (5 marks)
- ii) Property binding vs event binding (5 marks)

Answer:

i) **Component metadata:**

The `@Component` decorator configures the component:

- `selector`: HTML tag name for the component
- `template/templateUrl`: HTML view
- `styleUrls`: Component-specific CSS

ii) **Property vs Event binding:**

- **Property binding** (`[prop]="expr"`): Data flows from component to template
- **Event binding** (`(event)="handler()"`): Data flows from template to component (user interactions)

PRACTICE EXAM SET 2

Total Marks: 100

SECTION A: MULTIPLE CHOICE QUESTIONS (20 Marks)

(1 mark each)

1. Which of the following is NOT a valid XML syntax rule?
 - a) Tags are case-sensitive
 - b) Attribute values must be quoted
 - c) Elements must be properly nested
 - d) Multiple root elements are allowed

Answer: d

Explanation: Well-formed XML must have exactly one root element.

2. In JavaScript, what is the value of `[] + []`?

- a) `[]`
- b) `0`
- c) `""` (empty string)
- d) `undefined`

Answer: c

Explanation: Arrays are converted to strings when using the `+` operator; empty arrays become empty strings.

3. Which CSS display value makes an element behave like a block-level container but allows inline placement?

- a) `block`
- b) `inline`
- c) `inline-block`
- d) `flex`

Answer: c

4. In TypeScript, what does `--strictNullChecks` do?

- a) Disallows `null` entirely
- b) Makes `null` and `undefined` only assignable to their own types
- c) Converts `null` to `undefined`
- d) Removes null safety checks

Answer: b

5. What does XSLT stand for?

- a) XML Style List Transformations
- b) Extensible Stylesheet Language Transformations
- c) XML Schema Language Transformations
- d) Extensible Structured List Translations

Answer: b

6. In JavaScript, which loop iterates over the values of an iterable?

- a) `for...in`
- b) `for...of`
- c) `while`
- d) `do...while`

Answer: b

Explanation: `for...in` iterates over keys; `for...of` iterates over values.

7. Which of the following is a valid TypeScript tuple?

- a) `let t: [number, string] = [1, "hello"]`
- b) `let t: (number, string) = [1, "hello"]`
- c) `let t: {number, string} = [1, "hello"]`
- d) `let t: number|string = [1, "hello"]`

Answer: a

8. What is the purpose of CDATA sections in XML?

- a) To define custom data types
- b) To include text that should not be parsed as XML
- c) To compress XML data
- d) To link to external DTDs

Answer: b

9. In CSS, which pseudo-class selects an element when the mouse is hovering over it?

- a) `:active`
- b) `:focus`
- c) `:hover`
- d) `:visited`

Answer: c

10. What will `typeof []` return in JavaScript?

- a) `"array"`
- b) `"object"`
- c) `"list"`
- d) `"undefined"`

Answer: b

Explanation: Arrays are objects in JavaScript (another quirk!). Use `Array.isArray()` to check for arrays.

11. Which Bootstrap grid breakpoint targets screens $\geq 992\text{px}$?

- a) `sm`
- b) `md`
- c) `lg`
- d) `xl`

Answer: c

12. In TypeScript, what is structural typing based on?

- a) Class inheritance hierarchy
- b) Object shape (properties and methods)
- c) Explicit interface implementation
- d) Type names

Answer: b

13. Which JavaScript method converts a string to an integer?

- a) `parseFloat()`
- b) `parseInt()`
- c) `Number()`
- d) `toInteger()`

Answer: b

14. What does the `?` operator mean in TypeScript interface properties?

- a) The property is required
- b) The property is optional
- c) The property can be null
- d) The property is read-only

Answer: b

15. In XML, which is a correct self-closing element?

- a) `<img>`
- b) `<img/>`
- c) `</img>`
- d) `<img self-close="true">`

Answer: b

16. What is JavaScript hoisting?

- a) Moving function and variable declarations to the top of their scope
- b) Sorting arrays in ascending order
- c) Lifting DOM elements to the top of the page
- d) Converting numbers to strings

Answer: a

17. In TypeScript, which syntax is used for type assertions?

- a) `value.cast<Type>()`
- b) `(Type)value`
- c) `value as Type`
- d) `Type:value`

Answer: c

18. Which CSS property specifies the order of flex items?

- a) `order`
- b) `flex-order`

- c) `item-order`
- d) `sort`

Answer: a

19. In Angular, which directive is used for conditional rendering?

- a) `*ngFor`
- b) `*ngIf`
- c) `*ngSwitch`
- d) `*ngWhile`

Answer: b

20. Which protocol provides secure (encrypted) web communication?

- a) HTTP
- b) HTTPS
- c) FTP
- d) SMTP

Answer: b

SECTION B: STRUCTURAL QUESTIONS (40 Marks)

Question 1: Markup Languages (10 marks)

a) Explain the historical evolution from SGML → HTML → XHTML → HTML5. Highlight the key motivations for each transition. (6 marks)

Answer:

1. **SGML**: Parent meta-standard (too complex)
 2. **HTML**: Simplified SGML for web pages (forgiving syntax)
 3. **XHTML**: HTML reformulated as strict XML (enforce well-formedness)
 4. **HTML5**: Practical compromise between strictness and real-world usage
-

b) Differentiate between well-formed XML and valid XML. (4 marks)

Answer:

- **Well-formed XML**: Follows XML syntax rules (one root, proper nesting, etc.)
 - **Valid XML**: Well-formed AND conforms to a schema/DTD
-

Question 2: XML Schema (15 marks)

Consider the following XML document representing student data:

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student id="S001">
    <name>Alice Johnson</name>
    <age>22</age>
    <major>Computer Science</major>
    <gpa>3.8</gpa>
  </student>
  <student id="S002">
    <name>Bob Smith</name>
    <age>21</age>
    <major>Mathematics</major>
  </student>
</students>
```

a) Create an XML Schema (XSD) that validates this document. Ensure:

- `id` is a required attribute of `student`
- `gpa` is optional
- `age` is an integer between 18 and 100 (10 marks)

Answer:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="students">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="student" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="age" type="xs:integer" base="xs:integer">
                <xs:restriction base="xs:integer">
                  <xs:minInclusive value="18"/>
                  <xs:maxInclusive value="100"/>
                </xs:restriction>
              </xs:sequence>
              <xs:element name="major" type="xs:string"/>
              <xs:element name="gpa" type="xs:decimal"
minOccurs="0"/>
            </xs:complexType>
          </xs:element>
          <xs:attribute name="id" type="xs:string"
use="required"/>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

```
</xs:element>
</xs:schema>
```

b) Explain two advantages of XML Schema over DTD. (5 marks)

Answer:

1. **Data types:** XSD supports built-in data types (numbers, dates, etc.) and custom types
2. **Namespaces:** Better namespace support for mixing vocabularies

Question 3: JavaScript Fundamentals (15 marks)

a) Explain the concept of variable scope in JavaScript. Differentiate between global scope and function/local scope. (8 marks)

Answer:

- **Global scope:** Variables declared outside functions; accessible everywhere
- **Function/local scope:** Variables declared inside a function; only accessible within that function
- **var** = function-scoped; **let/const** = block-scoped

b) What is a closure in JavaScript? Provide a simple example. (7 marks)

Answer:

A closure is a function that retains access to its lexical scope even when executed outside it.

Example:

```
function counter() {
  let count = 0;
  return function() {
    return ++count;
  };
}

const c = counter();
console.log(c()); // 1
console.log(c()); // 2
```

SECTION C: ESSAY AND CODE QUESTIONS (40 Marks)

Question 1: DOM Manipulation and Events (20 marks)

Write a complete HTML, CSS, and JavaScript program for a simple to-do list application that:

a) Displays an input field to add new to-do items

- b) Has an "Add" button that adds the item to a list
- c) Each list item has a "Delete" button to remove it
- d) Uses event listeners (not inline event handlers)
- e) Applies basic styling with CSS (12 marks)

Answer:

```
<!DOCTYPE html>
<html>
<head>
  <style>
    body { font-family: Arial, sans-serif; max-width: 500px; margin: 2rem
auto; }
    li { margin: 0.5rem 0; padding: 0.5rem; background: #f5f5f5; border-
radius: 4px; }
    .delete { color: red; cursor: pointer; margin-left: 1rem; }
  </style>
</head>
<body>
  <h1>To-Do List</h1>
  <input type="text" id="todoInput" placeholder="New to-do">
  <button id="addBtn">Add</button>
  <ul id="todoList"></ul>

  <script>
    const todoInput = document.getElementById('todoInput');
    const addBtn = document.getElementById('addBtn');
    const todoList = document.getElementById('todoList');

    addBtn.addEventListener('click', addTodo);
    todoList.addEventListener('click', function(e) {
      if (e.target.classList.contains('delete')) {
        e.target.parentElement.remove();
      }
    });

    function addTodo() {
      const text = todoInput.value.trim();
      if (!text) return;

      const li = document.createElement('li');
      li.textContent = text;
      const deleteBtn = document.createElement('span');
      deleteBtn.className = 'delete';
      deleteBtn.textContent = 'Delete';
      li.appendChild(deleteBtn);
      todoList.appendChild(li);
      todoInput.value = '';
    }
  </script>
```

```
</body>
</html>
```

f) Explain how event delegation could be used to optimize this application if you had thousands of to-do items. (8 marks)

Answer:

Instead of attaching a delete listener to every single list item, attach one listener to the parent `<ul>`. The event bubbles up, and we check which element was clicked. This is much more efficient for large numbers of elements.

Question 2: TypeScript Programming (20 marks)

a) Create a TypeScript class `BankAccount` with:

- Private properties: `accountNumber` (string), `balance` (number)
- A constructor that initializes both properties
- A public method `deposit(amount: number)` that adds to the balance
- A public method `withdraw(amount: number)` that subtracts from the balance (only if sufficient funds)
- A public method `getBalance()` that returns the current balance

Write code to demonstrate creating an instance and using all methods. (12 marks)

Answer:

```
class BankAccount {
    private accountNumber: string;
    private balance: number;

    constructor(accNum: string, initialBalance: number) {
        this.accountNumber = accNum;
        this.balance = initialBalance;
    }

    deposit(amount: number): void {
        this.balance += amount;
    }

    withdraw(amount: number): boolean {
        if (amount <= this.balance) {
            this.balance -= amount;
            return true;
        }
        return false;
    }

    getBalance(): number {
        return this.balance;
    }
}
```

```
}

// Usage
const myAccount = new BankAccount("12345", 1000);
myAccount.deposit(500);
console.log(myAccount.getBalance()); // 1500
myAccount.withdraw(300);
console.log(myAccount.getBalance()); // 1200
```

b) Then create a subclass `SavingsAccount` that:

- Extends `BankAccount`
- Adds an `interestRate` property
- Adds an `applyInterest()` method that adds interest to the balance (8 marks)

Answer:

```
class SavingsAccount extends BankAccount {
  private interestRate: number;

  constructor(accNum: string, initialBalance: number, rate: number) {
    super(accNum, initialBalance);
    this.interestRate = rate;
  }

  applyInterest(): void {
    const interest = this.getBalance() * this.interestRate;
    this.deposit(interest);
  }
}
```

PRACTICE EXAM SET 3

Total Marks: 100

SECTION A: MULTIPLE CHOICE QUESTIONS (20 Marks)

(1 mark each)

1. What is the output of `2 + '2'` in JavaScript?

- a) 4
- b) 22
- c) NaN
- d) undefined

Answer: b

Explanation: String concatenation! When `+` has a string operand, it concatenates instead of adding.

2. In CSS, which property creates rounded corners?

- a) `corner-radius`
- b) `border-round`
- c) `border-radius`
- d) `round-corner`

Answer: c

3. Which of the following is NOT a valid JavaScript data type?

- a) `undefined`
- b) `symbol`
- c) `float`
- d) `bigint`

Answer: c

Explanation: JavaScript has `number` (covers both integers and floats), not a separate `float` type.

4. In TypeScript, what is a generic?

- a) A function that works with multiple types while preserving type information
- b) A type that can be any value
- c) A class without properties
- d) A function with no parameters

Answer: a

5. What is the default Bootstrap grid system based on?

- a) 10 columns
- b) 12 columns
- c) 16 columns
- d) 24 columns

Answer: b

6. In JavaScript, what does `==` do compared to `===`?

- a) `==` compares values; `===` compares values and types
- b) `===` compares values; `==` compares values and types
- c) They are identical
- d) `==` is for strings; `===` is for numbers

Answer: a

Explanation: Always use `===` unless you explicitly want type coercion!

7. Which XML technology is used to navigate through elements and attributes in an XML document?

- a) XSLT
- b) XPath

- c) XQuery
- d) XSD

Answer: b

8. In TypeScript, which keyword is used for subclass inheritance?

- a) `inherits`
- b) `extends`
- c) `implements`
- d) `derives`

Answer: b

9. What does `document.getElementById()` return if no element is found?

- a) `null`
- b) `undefined`
- c) An empty object
- d) Throws an error

Answer: a

10. Which CSS position value removes an element from the normal document flow?

- a) `static`
- b) `relative`
- c) `absolute`
- d) `sticky`

Answer: c

11. In TypeScript, what is the `any` type?

- a) A type that represents no values
- b) A type that disables type checking for that value
- c) A type that can only be numbers
- d) A type that is only for arrays

Answer: b

Explanation: Avoid overusing `any` - it defeats the purpose of TypeScript!

12. What is a web server?

- a) A client program that requests web pages
- b) A program that stores and delivers web pages over HTTP
- c) A database server
- d) An email server

Answer: b

13. In JavaScript, which method adds elements to the end of an array?

- a) `push()`
- b) `pop()`
- c) `shift()`
- d) `unshift()`

Answer: a

14. In XML DTD, what does `#PCDATA` mean?

- a) Private Character Data
- b) Parsed Character Data
- c) Protected Character Data
- d) Primary Character Data

Answer: b

15. Which Bootstrap utility class sets margin-top to 1rem?

- a) `mt-1`
- b) `mt-2`
- c) `mt-3`
- d) `mt-4`

Answer: c

Explanation: `mt-3` = margin-top of 1rem (16px) in Bootstrap.

16. In TypeScript, what is the `readonly` modifier used for?

- a) To make properties only readable after initialization
- b) To make classes immutable
- c) To prevent method overriding
- d) To hide properties

Answer: a

17. What is the output of `console.log(0.1 + 0.2 === 0.3)` in JavaScript?

- a) `true`
- b) `false`
- c) `NaN`
- d) `undefined`

Answer: b

Explanation: Floating-point precision issue! $0.1 + 0.2 = 0.30000000000000004$ in JavaScript.

18. Which CSS selector has the highest specificity?

- a) Element selector
- b) Class selector

- c) ID selector
- d) Universal selector

Answer: c

19. In Angular, what does `{{ expression }}` syntax represent?

- a) Property binding
- b) Event binding
- c) Interpolation
- d) Two-way binding

Answer: c

20. Which protocol is used for email transmission?

- a) HTTP
- b) FTP
- c) SMTP
- d) SSH

Answer: c

SECTION B: STRUCTURAL QUESTIONS (40 Marks)

Question 1: HTTP and Web Architecture (10 marks)

a) Describe the HTTP request-response cycle. Include the roles of client, server, and the typical flow. (6 marks)

Answer:

1. Client (browser) sends HTTP request to server
 2. Server processes request
 3. Server sends HTTP response back to client
 4. Client displays response
-

b) Explain the difference between persistent and non-persistent HTTP connections. (4 marks)

Answer:

- **Non-persistent:** New TCP connection for every request/response
 - **Persistent:** Reuse same TCP connection for multiple requests
-

Question 2: CSS Selectors and Specificity (15 marks)

Given the following HTML:

```
<!DOCTYPE html>
<html>
```

```
<head>
  <style>
    #special { color: blue; }
    .highlight { color: red; }
    p { color: green; }
    div p { color: purple; }
  </style>
</head>
<body>
  <div>
    <p id="special" class="highlight">Hello World!</p>
  </div>
</body>
</html>
```

a) What color will "Hello World!" be displayed in? Explain your answer by discussing CSS specificity. (8 marks)

Answer:

- **Blue**
- Specificity order: ID selector > Class selector > Element selector
- `#special` (ID) has the highest specificity

b) List the CSS selectors in order from highest specificity to lowest. (7 marks)

Answer:

1. `#special` (ID)
2. `div p` (element + element)
3. `.highlight` (class)
4. `p` (element)

Question 3: JavaScript Arrays and Objects (15 marks)

a) Given the array:

```
let fruits = ['apple', 'banana', 'cherry', 'date'];
```

Show how to:

- i) Add 'elderberry' to the end (2 marks)
- ii) Remove the first element (2 marks)
- iii) Find the index of 'cherry' (2 marks)
- iv) Create a new array with only fruits starting with 'b' (3 marks)

Answer:

- i) `fruits.push('elderberry');`
- ii) `fruits.shift();`


```

iii) fruits.indexOf('cherry');
iv) fruits.filter(f => f.startsWith('b'));

```

b) Explain the difference between primitive types and objects in JavaScript, especially regarding how they are passed to functions. (6 marks)

Answer:

- **Primitives:** Passed by value (copy made)
- **Objects:** Passed by reference (function modifies original)

SECTION C: ESSAY AND CODE QUESTIONS (40 Marks)

Question 1: JavaScript Dice Simulator (20 marks)

Write a JavaScript program (with HTML and CSS) that simulates rolling two dice:

- Display two dice images (you can use placeholder images or text representations)
- Have a "Roll Dice" button that generates two random numbers between 1 and 6
- Display the corresponding dice faces
- Keep track of and display the roll history (last 10 rolls)
- Calculate and display the frequency of each possible sum (2-12) after each roll (15 marks)

Answer:

```

<!DOCTYPE html>
<html>
<head>
  <style>
    .dice { font-size: 4rem; margin: 1rem; }
    body { font-family: Arial, sans-serif; max-width: 600px; margin: 2rem
auto; text-align: center; }
  </style>
</head>
<body>
  <h1>Dice Simulator</h1>
  <div class="dice" id="die1"></div>
  <div class="dice" id="die2"></div>
  <button id="rollBtn">Roll Dice</button>
  <h3>History (Last 10):</h3>
  <div id="history"></div>
  <h3>Sum Frequencies:</h3>
  <div id="freq"></div>

  <script>
    const diceChars = ['❶', '❷', '❸', '❹', '❺', '❻'];
    const die1 = document.getElementById('die1');

```

```

const die2 = document.getElementById('die2');
const historyDiv = document.getElementById('history');
const freqDiv = document.getElementById('freq');
let history = [];
let frequencies = {};

document.getElementById('rollBtn').addEventListener('click', roll);

function roll() {
  const d1 = Math.floor(Math.random() * 6) + 1;
  const d2 = Math.floor(Math.random() * 6) + 1;
  const sum = d1 + d2;

  die1.textContent = diceChars[d1 - 1];
  die2.textContent = diceChars[d2 - 1];

  history.unshift(`${d1} + ${d2} = ${sum}`);
  if (history.length > 10) history.pop();

  frequencies[sum] = (frequencies[sum] || 0) + 1;

  historyDiv.innerHTML = history.join('<br>');
  freqDiv.innerHTML = Object.entries(frequencies)
    .map(([s, c]) => `Sum ${s}: ${c}`)
    .join('<br>');
}
</script>
</body>
</html>

```

f) Explain how you would modify this to roll N dice instead of just 2. (5 marks)

Answer:

Use a loop to generate N random numbers, store them in an array, then dynamically create dice elements for each one.

Question 2: TypeScript Interfaces and Generics (20 marks)

a) Define a generic TypeScript interface `Repository<T>` with:

- A method `add(item: T): void`
- A method `getAll(): T[]`
- A method `findById(id: number): T | undefined`

Then implement this interface for a `UserRepository` where `T` is a `User` interface with `id` (number), `name` (string), and `email` (string). (12 marks)

Answer:

```
interface User {
  id: number;
  name: string;
  email: string;
}

interface Repository<T> {
  add(item: T): void;
  getAll(): T[];
  findById(id: number): T | undefined;
}

class UserRepository implements Repository<User> {
  private users: User[] = [];

  add(user: User): void {
    this.users.push(user);
  }

  getAll(): User[] {
    return [...this.users];
  }

  findById(id: number): User | undefined {
    return this.users.find(u => u.id === id);
  }
}
```

b) Write a generic function `identity<T>(arg: T): T` that returns the argument it receives. Explain why generics are useful here instead of just using `any`. (8 marks)

Answer:

```
function identity<T>(arg: T): T {
  return arg;
}
```

- With `any`, type information is lost
- With generics, TypeScript knows the return type is same as argument type
- Better type safety and autocompletion

PRACTICE EXAM SET 4

Total Marks: 100

SECTION A: MULTIPLE CHOICE QUESTIONS (20 Marks)

(1 mark each)

1. What is the output of `!''` in JavaScript?

- a) `true`
- b) `false`
- c) `''`
- d) `undefined`

Answer: a

Explanation: Empty string is falsy; `!` converts to boolean and negates it.

2. In CSS, which property changes the transparency of an element?

- a) `opacity`
- b) `transparency`
- c) `alpha`
- d) `visibility`

Answer: a

3. Which of the following is true about TypeScript?

- a) It runs directly in browsers
- b) It is a superset of JavaScript
- c) It removes all JavaScript features
- d) It is only for backend development

Answer: b

4. In XML, which entity reference represents an ampersand?

- a) `&`
- b) `<`
- c) `>`
- d) `"`

Answer: a

5. In JavaScript, what does `array.map()` do?

- a) Filters elements based on a condition
- b) Creates a new array by applying a function to each element
- c) Reduces the array to a single value
- d) Sorts the array

Answer: b

6. Which TypeScript feature allows a class to provide an implementation for an abstract method?

- a) Overloading
- b) Overriding

- c) Implementing
- d) Extending

Answer: b

7. What is the purpose of the viewport meta tag in Bootstrap?

- a) To set the page title
- b) To enable responsive behavior on mobile devices
- c) To link to CSS files
- d) To define the page language

Answer: b

8. In JavaScript, which statement creates a block-scoped variable?

- a) `var`
- b) `let`
- c) `const`
- d) Both `let` and `const`

Answer: d

9. Which XML schema language is written in XML itself?

- a) DTD
- b) XML Schema (XSD)
- c) RELAX NG
- d) Schematron

Answer: b

10. In CSS, `flexbox` is primarily used for:

- a) Creating tables
- b) One-dimensional layouts
- c) Two-dimensional layouts
- d) Animations

Answer: b

Explanation: Flexbox = 1D, Grid = 2D.

11. What is the output of `typeof NaN`?

- a) `"number"`
- b) `"NaN"`
- c) `"undefined"`
- d) `"object"`

Answer: a

Explanation: Another quirk! NaN stands for "Not a Number", but its type is number.

12. In TypeScript, which access modifier makes a property accessible only within its own class?

- a) `public`
- b) `private`
- c) `protected`
- d) `internal`

Answer: b

13. Which JavaScript event fires when the DOM is fully loaded?

- a) `window.onload`
- b) `DOMContentLoaded`
- c) `document.ready`
- d) `load`

Answer: b

14. In XML DTD, what does `+` mean in a content model?

- a) Zero or one occurrence
- b) Exactly one occurrence
- c) Zero or more occurrences
- d) One or more occurrences

Answer: d

15. Which Bootstrap class makes text white?

- a) `text-primary`
- b) `text-secondary`
- c) `text-white`
- d) `text-light`

Answer: c

16. In TypeScript, what are rest parameters used for?

- a) To pass a variable number of arguments to a function
- b) To define optional parameters
- c) To set default parameter values
- d) To skip parameters

Answer: a

17. What is the output of `5 < 3 < 2` in JavaScript?

- a) `true`
- b) `false`
- c) `NaN`
- d) `undefined`

Answer: a

Explanation: $5 \ \&\& \ 3 \rightarrow \text{false}$, then $\text{false} \ \&\& \ 2 \rightarrow \text{true}$ (because `false` is 0 when converted to number).

18. Which CSS property controls the order of elements in a flex container?

- a) `flex-direction`
- b) `order`
- c) `flex-wrap`
- d) `justify-content`

Answer: a

19. In Angular, which directive is used to iterate over an array?

- a) `*ngIf`
- b) `*ngFor`
- c) `*ngSwitch`
- d) `*ngRepeat`

Answer: b

20. Which protocol is used for file transfer?

- a) HTTP
- b) SMTP
- c) FTP
- d) SSH

Answer: c

SECTION B: STRUCTURAL QUESTIONS (40 Marks)

Question 1: Type System Comparison (10 marks)

a) Compare and contrast JavaScript and TypeScript type systems. (6 marks)

Answer:

- **JavaScript:** Dynamically typed; types checked at runtime
 - **TypeScript:** Statically typed superset of JavaScript; types checked at compile time
-

b) Explain the benefits of using TypeScript over JavaScript for large-scale applications. (4 marks)

Answer:

1. Catches errors at compile time
2. Better IDE support (autocompletion, refactoring)
3. More maintainable code
4. Self-documenting types

Question 2: CSS Box Model (15 marks)

a) Explain the CSS box model. Include content, padding, border, and margin in your explanation. (8 marks)

Answer:

- **Content:** Actual element content (text, images)
- **Padding:** Space inside border
- **Border:** Goes around padding
- **Margin:** Space outside border

b) Given an element with:

- Width: 200px
- Padding: 20px
- Border: 5px solid black
- Margin: 10px

Calculate the total width the element occupies on the page (using the standard box model). (7 marks)

Answer:

Total width = width + 2padding + 2border + 2*margin
= 200 + 40 + 10 + 20
= 270px

Question 3: JavaScript Async and Events (15 marks)

a) Explain the concept of the event loop in JavaScript. How does it handle asynchronous operations? (8 marks)

Answer:

1. Call stack executes sync code
2. Async operations go to Web APIs
3. When ready, callbacks go to task queue
4. Event loop pushes callbacks to call stack when empty

b) Differentiate between `alert()`, `confirm()`, and `prompt()` in JavaScript. (7 marks)

Answer:

- `alert()`: Shows message, OK button
- `confirm()`: Shows message, OK/Cancel; returns boolean
- `prompt()`: Shows message, input field; returns input string

SECTION C: ESSAY AND CODE QUESTIONS (40 Marks)**Question 1: Temperature Converter (20 marks)**

Create a complete web application that converts temperatures between Celsius, Fahrenheit, and Kelvin:

- a) Display three input fields (Celsius, Fahrenheit, Kelvin)
- b) When the user enters a value in any field, the other two fields update automatically
- c) Handle invalid inputs gracefully
- d) Use event listeners and DOM manipulation
- e) Apply styling with Bootstrap for a clean, responsive interface (15 marks)

Answer:

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body { max-width: 500px; margin: 3rem auto; }
  </style>
</head>
<body class="container">
  <h1 class="mb-4">Temperature Converter</h1>
  <div class="mb-3">
    <label class="form-label">Celsius</label>
    <input type="number" id="celsius" class="form-control">
  </div>
  <div class="mb-3">
    <label class="form-label">Fahrenheit</label>
    <input type="number" id="fahrenheit" class="form-control">
  </div>
  <div class="mb-3">
    <label class="form-label">Kelvin</label>
    <input type="number" id="kelvin" class="form-control">
  </div>

  <script>
    const cInput = document.getElementById('celsius');
    const fInput = document.getElementById('fahrenheit');
    const kInput = document.getElementById('kelvin');

    cInput.addEventListener('input', () => {
      const c = parseFloat(cInput.value);
      if (isNaN(c)) return;
      fInput.value = (c * 9/5 + 32).toFixed(2);
      kInput.value = (c + 273.15).toFixed(2);
    });

    fInput.addEventListener('input', () => {
```

```
        const f = parseFloat(fInput.value);
        if (isNaN(f)) return;
        const c = (f - 32) * 5/9;
        cInput.value = c.toFixed(2);
        kInput.value = (c + 273.15).toFixed(2);
    });

    kInput.addEventListener('input', () => {
        const k = parseFloat(kInput.value);
        if (isNaN(k)) return;
        const c = k - 273.15;
        cInput.value = c.toFixed(2);
        fInput.value = (c * 9/5 + 32).toFixed(2);
    });
</script>
</body>
</html>
```

f) Explain how you would extend this to include more temperature scales. (5 marks)

Answer:

Add more input fields, create conversion functions between all pairs, and attach event listeners to update all other fields when any field changes.

Question 2: TypeScript and Angular Integration (20 marks)

a) Create a TypeScript class `Task` with:

- `id`: number
- `title`: string
- `completed`: boolean
- `dueDate`: Date

Include a constructor and appropriate methods. (8 marks)

Answer:

```
class Task {
    id: number;
    title: string;
    completed: boolean;
    dueDate: Date;

    constructor(id: number, title: string, dueDate: Date) {
        this.id = id;
        this.title = title;
        this.completed = false;
        this.dueDate = dueDate;
    }
}
```

```
toggleComplete(): void {
    this.completed = !this.completed;
}

isOverdue(): boolean {
    return this.dueDate < new Date() && !this.completed;
}
}
```

b) Briefly describe how you would use this `Task` class in an Angular component to display a list of tasks. Include:

- Component structure
- Template with `*ngFor`
- Property binding (12 marks)

Answer:

1. Create a component with a `tasks` property of type `Task[]`
2. In the template, use `*ngFor="let task of tasks"` to loop through them
3. Use property binding to display task data and bind events

Example template snippet:

```
<ul>
  <li *ngFor="let task of tasks" [class.overdue]="task.isOverdue()">
    {{ task.title }} - {{ task.dueDate | date }}
    <button (click)="task.toggleComplete()">
      {{ task.completed ? 'Undo' : 'Complete' }}
    </button>
  </li>
</ul>
```

General Study Tips

- **JavaScript quirks:** Remember `typeof null === "object"`, `0.1 + 0.2 !== 0.3`, `[] + [] === ""`
- **Always use `===`:** Unless you explicitly want type coercion
- **Numeric sort:** Use comparator function `(a, b) => a - b`
- **XML well-formedness:** Exactly one root, proper nesting, quoted attributes
- **TypeScript best practices:** Avoid `any`, use interfaces, use generics for reusability

Good luck with your exams!